

Integrated Insurance Policy and Claims Claims Platform

Scenario:

Multi-Line Insurance Policy and Claims Management solution is developed to offer a unified Salesforce-based solution for handling Vehicle, Property, and Life Insurance policies and claims. The solution enables insurance agents to build and handle policy details using a flexible data model with policy-specific Record Types and Field Sets. Guided Screen Flows simplify policy quotation, while Apex automated processing calculates premiums based on policy and vehicle details. Claims are automatically routed to the relevant queue based on policy type, and large-value claims are processed through an approval flow involving Senior Adjusters and Claims Managers. An LWC-based Claims Dashboard offers adjusters with a focused view of their assigned claims, including claim amount, policy type, policy holder, status, and days open. This initiative enhances policy processing efficiency, claim handling, data consistency, operational view, and policyholder enable.

1. INTRODUCTION

1.1 Solution Overview :

The Multi-Line Insurance Policy and Claims Management solution is a Salesforce-based solution developed to unify and simplify the management of Vehicle, Property, and Life Insurance policies and claims. The solution replaces disconnected and manual-based workflows with structured Salesforce objects, Record Types, Field Sets, Flows, Apex automated processing, Approval workflows, and Lightning Web Components.

The solution enables insurance agents to build and handle policies through a flexible data model, while guided Screen Flows simplify the policy quotation approval flow. The solution validates policy details and uses Apex automated processing to compute premiums based on factors such as state and model year. Once a claim is generated, automated processing identifies the connected policy type and routes the claim to the relevant claims queue.

For claims adjusters, an LWC-based Claims Dashboard offers view into assigned claims, including claim number, policy type, policy holder, claim amount, approval status, and days open. large-value claims exceeding the defined threshold are processed through a structured approval approval flow involving Senior Adjusters and Claims Managers. access control and access are managed using sharing rules and role-specific Permission Sets.

Overall, the initiative offers a unified solution for enhancing policy processing, premium computation, claim routing, claim approval, data consistency, operational view, and policyholder enable across multiple insurance lines.

1.2 Purpose : The purpose of the Multi-Line Insurance Policy and Claims Management System is to provide a centralized Salesforce-based platform for managing Vehicle, Property, and Life Insurance policies and claims. The system is designed to overcome the limitations of legacy processes that result in manual, slow, and inconsistent quoting, policy issuance, and claims handling. The solution aims to standardize and accelerate policy quoting and issuance, automate claim initiation and routing based on policy type and claim severity, and provide a 360-degree view of Customer, Policy, and Claim information. It also supports guided quoting through Screen Flows, automated premium and claim processing, data validation, and an LWC-based dashboard to improve claims adjuster efficiency.

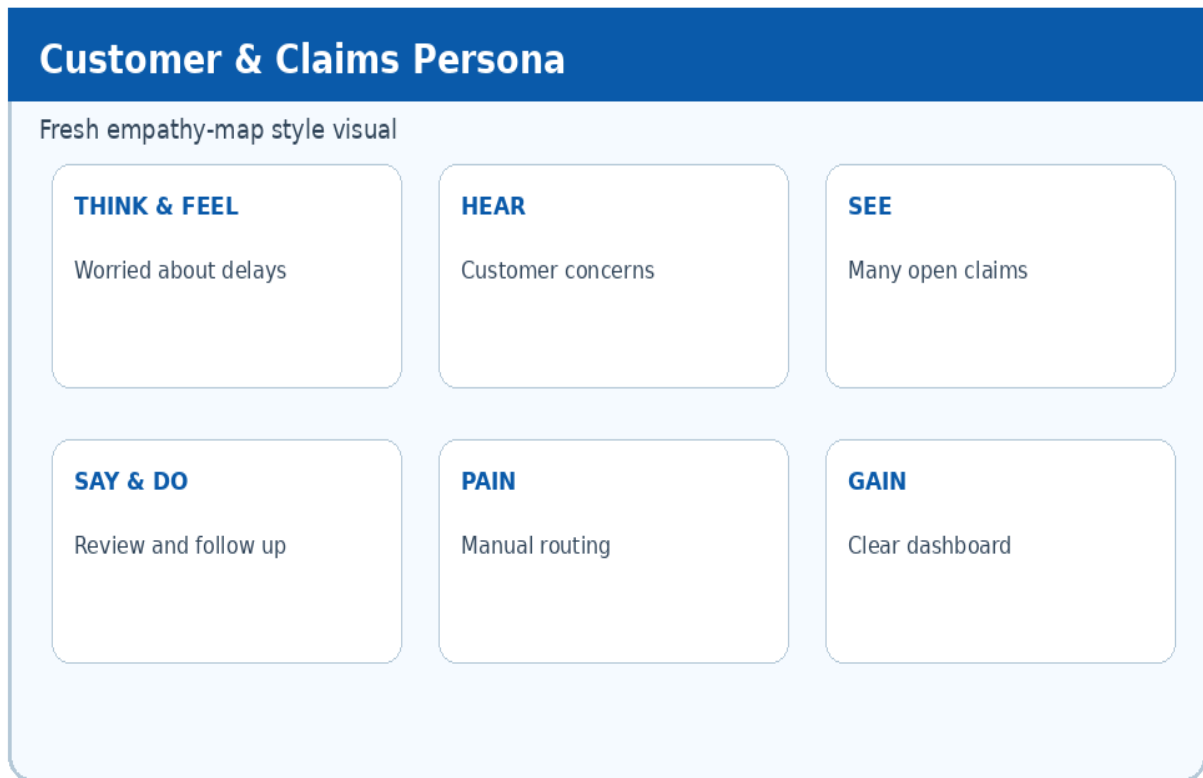
Overall, the initiative focuses on enhancing operational efficiency, data consistency, claims processing, and policyholder experience by bringing multiple insurance lines into a unified Salesforce solution.

2. IDEATION PHASE

2.1 Challenge Statement (Customer View Template)

PS I am (Customer)	I'm trying to	But Because Which makes me feel		
PS-1 Insurance Operations Manager	standardize policy quotation and issuance	the approval flow is manual-based, slow, and inconsistent	older systems and workflows are disconnected	frustrated and concerned
PS-2 Insurance Agent	build and handle policies effectively	different insurance lines require different details	Vehicle, Property, and Life policies have diverse data requirements	overwhelmed
PS-3 Insurance Agent	offer accurate policy quotes	quoting requires multiple steps and manual-based processing	there is no unified quoting solution	frustrated and uncertain
PS-4 Claims Manager	handle claims effectively	claim review and routing are largely manual-based	claims workflows are not unified or solution-driven	stressed

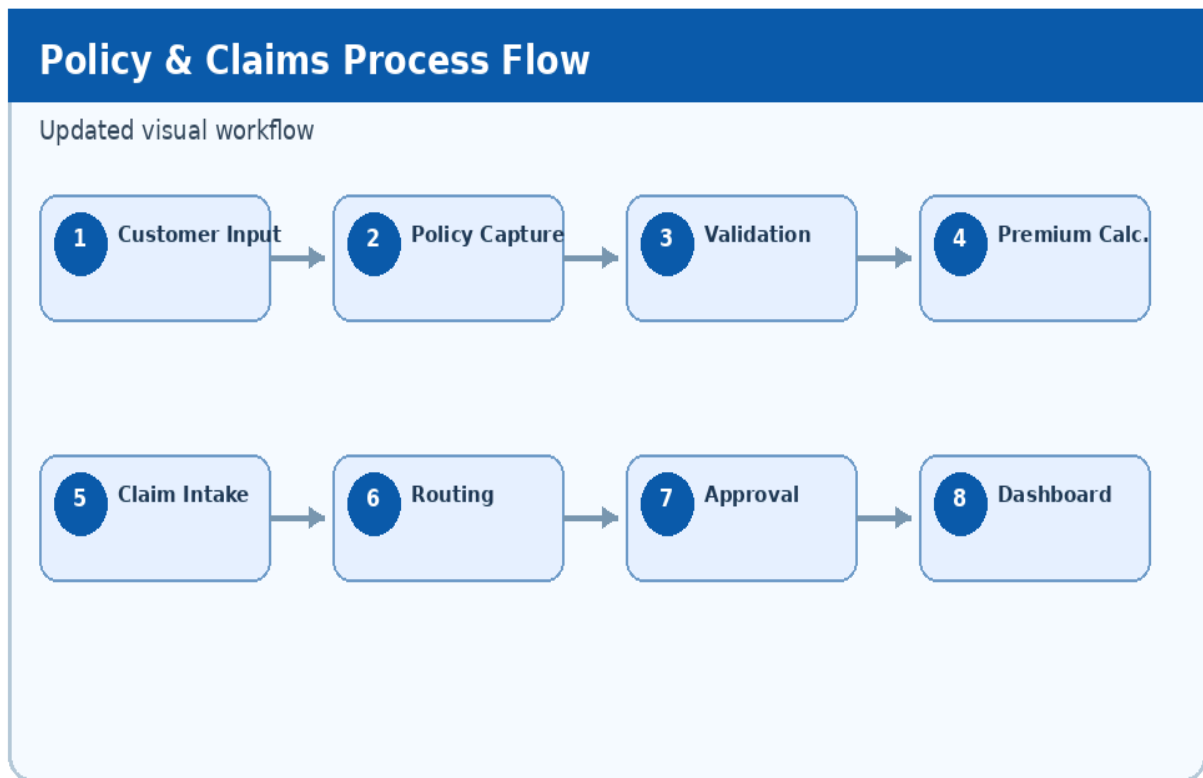
PS-5 Claims Adjuster	handle assigned claims effectively	I lack a focused view of my claims workload	claim details is not easily available in one place	frustrated and less productive
PS-6 Claims Manager	route claims to the relevant team	claims need to be handled according to policy type and claim severity	routing workflows are complex and require automated processing	concerned about delays
PS-7 Senior Adjuster approval flow	large-value claims effectively	approval requires multiple levels of review	large-value claims require structured approval and escalation	pressured
PS-8 Insurance Operations Manager	maintain accurate policy details	different products require different fields and validations	Vehicle, Property, and Life policies have varying data requirements	concerned about data quality
PS-9 Customer get my policy and claim handled quickly		manual-based processing causes delays	policy and claims workflows are disconnected	uncertain and dissatisfied
PS-10 Insurance Management	get a complete view of policyholder, policy, and claim details	details is spread across workflows and systems	there is no unified solution	concerned about operational efficiency



2.2 User Empathy Canvas

User Persona: Insurance Claims Adjuster

- Think & Feel: Concerned about claim delays and workload
- Hear: Customer and team concerns about claim resolution
- See: Multiple claims, policy details, and approval statuses
- Say & Do: Reviews claims and follows up on approvals
- Pain: manual-based processing, complex routing & limited view
- Gain: Claims control panel, automated processing & faster approvals



2.3 Idea Generation & Prioritization

Ideas Considered

- manual-based Registers ? Policy and claim details maintained manually - Spreadsheet Tracking ? Policies, claims, and approvals tracked using spreadsheets - Standalone Portal ? Separate portal for policy and claim management - Salesforce CRM ? Unified solution for policies, claims, automated processing, and reports

Final Selection

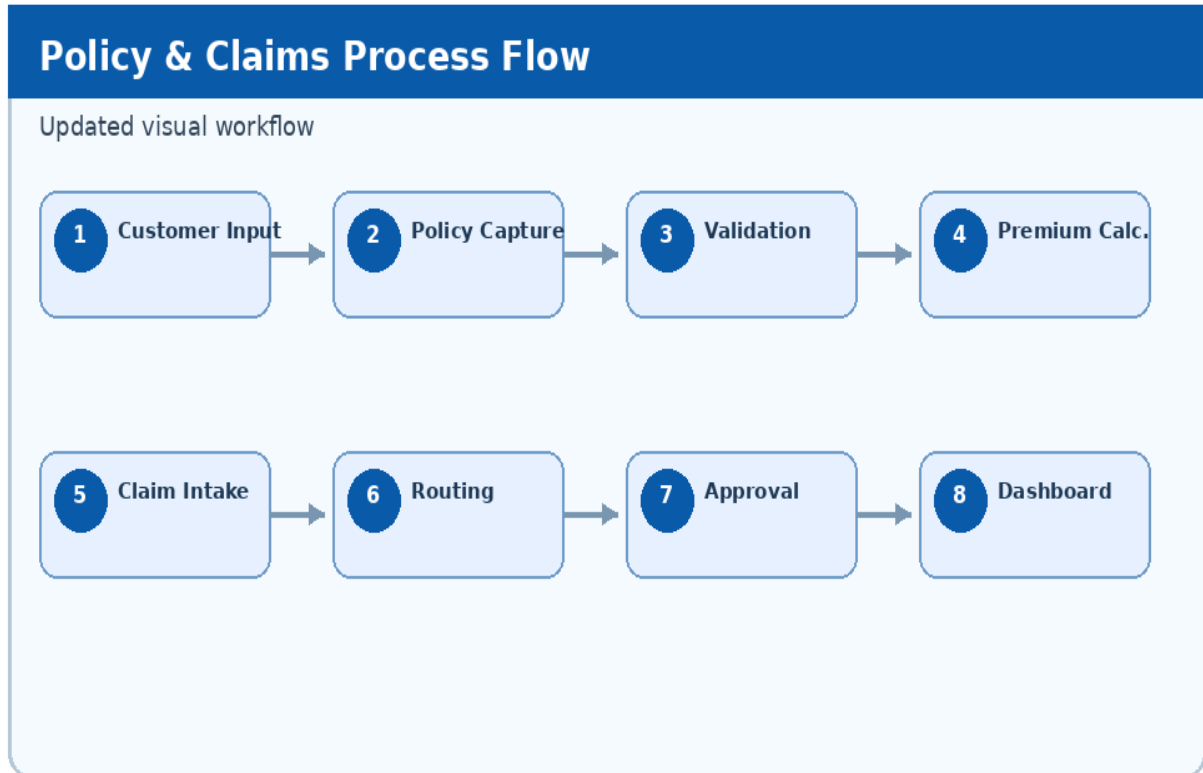
Salesforce CRM chosen due to:

- automated processing ? Automates quoting, premium computation, claim routing, and approval workflows - Scalability ? enables multiple insurance lines including Vehicle, Property, and

Life Insurance

- Real-Time view ? offers unified view into policyholder, policy, and claim details
- Unified solution ? Brings policy and claims workflows together in one solution
- enhanced Efficiency ? Reduces manual-based processing and enables faster claim

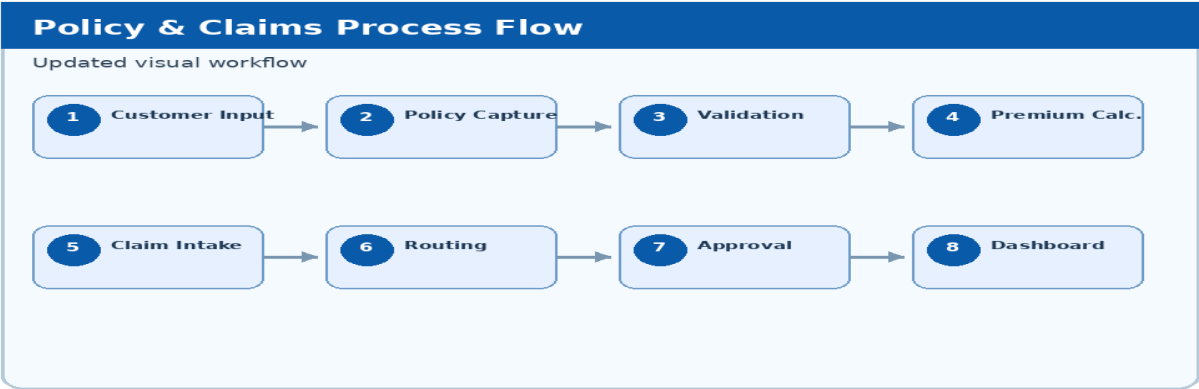
handling



Stages:

?? Insurance Inquiry ?? Policy Selection ?
Vehicle / Property / Life ?? Quote
Request

?? Policy Information
Capture ?? Premium
Calculation ?? Policy
Issuance ?? Claim Initiation
?? Claim Routing ?? Claim
Review & Approval ??
Claim Resolution ??
Customer Confirmation ??
Reporting & Visibility



3.2 System Requirements

Core Functional Requirements

FR No Functional need	
FR-01 The solution must enable users to build and handle Insurance Policy records for Vehicle, Property, and Life Insurance.	

FR-02 The solution must enable separate Record Types for Vehicle, Property, and Life policies.
FR-03 The solution must record product-specific policy details such as VIN for Vehicle Insurance and Square Footage/Year Built for Property Insurance.
FR-04 The solution must offer a guided Auto Quoting Screen Flow to record policy holder, policy start date, state, VIN, and model year.
FR-05 The solution must build a draft policy during the quoting approval flow.
FR-06 The solution must compute policy premiums using Apex-based premium computation logic.
FR-07 The solution must check Vehicle Insurance VINs and confirm that the VIN contains exactly 17 characters.
FR-08 The solution must enable users to build and handle Claim records linked with policies.
FR-09 The solution must enable different Claim Record Types for Accident, Property, and Life claims.
FR-10 The solution must automatically route claims to the relevant queue based on the connected policy type.
FR-11 The solution must identify large-value claims based on the defined claim amount threshold.
FR-12 The solution must submit qualifying large-value claims through a multi-level approval approval flow involving Senior Adjusters and Claims Managers.
FR-13 The solution must offer an Approve/Reject Claim capability with approver comments.
FR-14 The solution must offer Claims Adjusters with an LWC Claims Dashboard showing assigned claims.

FR-15 The control panel must display details such as Claim Number, Policy Type, Policy Holder, Claim Amount, Status, and Days Open.
FR-16 The control panel must enable users to filter claims by All, Auto, Property, and Life.
FR-17 The solution must offer solution-driven updates to connected Policy and Claim records through Record-Triggered Flows.
FR-18 The solution must enable state-based access/sharing for applicable insurance users and records.
FR-19 The solution must offer role-specific access through Permission Sets for Insurance Agents, Claims Adjusters, and Claims Managers.
FR-20 The solution must enable reports and view of Customer, Policy, and Claim details from the unified Salesforce solution.

Non-Core Functional Requirements

The following requirements describe the expected quality, usability, access control, scalability, and maintainability of the solution. Some are derived requirements based on the initiative's stated objectives and challenges rather than explicitly documented as formal NFRs.

ID Non-Functional need	Description
NFR-01 Usability	The solution is expected to offer simple, guided Screen Flows that minimize unnecessary clicks and make policy quotation easy for agents.
NFR-02 Performance	Policy quoting, premium computation, claim routing, and control panel operations is expected to execute effectively to enable day-to-day insurance operations.

ID Non-Functional need	Description
	NFR-03 Scalability The solution is expected to enable the three insurance lines?Vehicle, Property, and Life?and enable additional insurance products to be accommodated in the future.
	NFR-04 Data Integrity Policy and claim details is expected to be validated to maintain accurate and consistent data across insurance products.
	NFR-05 access control Access to insurance details is expected to be controlled according to the user's responsibility, including state-based sharing and role-specific Permission Sets.
	NFR-06 Maintainability The solution is expected to use modular Salesforce components such as Flows, Apex classes, LWC components, Record Types, and Field Sets so that individual components can be maintained independently.
	NFR-07 Reliability solution-driven claim routing, premium computation, and approval workflows is expected to consistently execute according to the defined business rules.
	NFR-08 Extensibility The data model is expected to accommodate different product-specific fields without requiring completely separate structures for each insurance line.
	NFR-09 view users is expected to have timely access to relevant Customer, Policy, and Claim details through unified Salesforce records and the Claims Dashboard.
	NFR-10 User Adoption The application is expected to offer an intuitive experience that reduces manual-based effort and enables effective adoption by agents and claims users.

3.3 Process Data Flow

Flow:

Customer / Agent ? Screen Flow ? Policy checking ? Policy__c ? PremiumCalculator ? Claim__c ? Claim Routing Flow ? Claims Queue ? Claims Adjuster ? Approval approval flow ? Claim Update ? LWC control panel / Reports

Main Data Flow

Step 1 ? Customer / Agent Input

? Customer and policy details is provided by the Insurance Agent.

Step 2 ? Policy Data record

? AutoQuoting Screen Flow records policy holder, policy start date, state, VIN, and model year.

Step 3 ? Policy checking

? Salesforce validates the entered details, including the 17-character VIN need for Auto policies.

Step 4 ? Policy Creation

? Valid details is stored in the Policy__c object with the relevant Vehicle, Property, or Life Record Type.

Step 5 ? Premium computation

? Policy details is passed to PremiumCalculator Apex to compute the applicable premium.

Step 6 ? Claim Creation

? When a claim occurs, the user creates a Claim__c record linked to the relevant Policy.

Step 7 ? Claim Routing

? A Record-Triggered Flow identifies the policy type and routes the claim to the relevant Auto, Property, or Life Claims Queue.

Step 8 ? Claim Review

? The Claims Adjuster accesses assigned claims through the Claims Dashboard and reviews policy and claim details.

Step 9 ? large-value Claim Approval

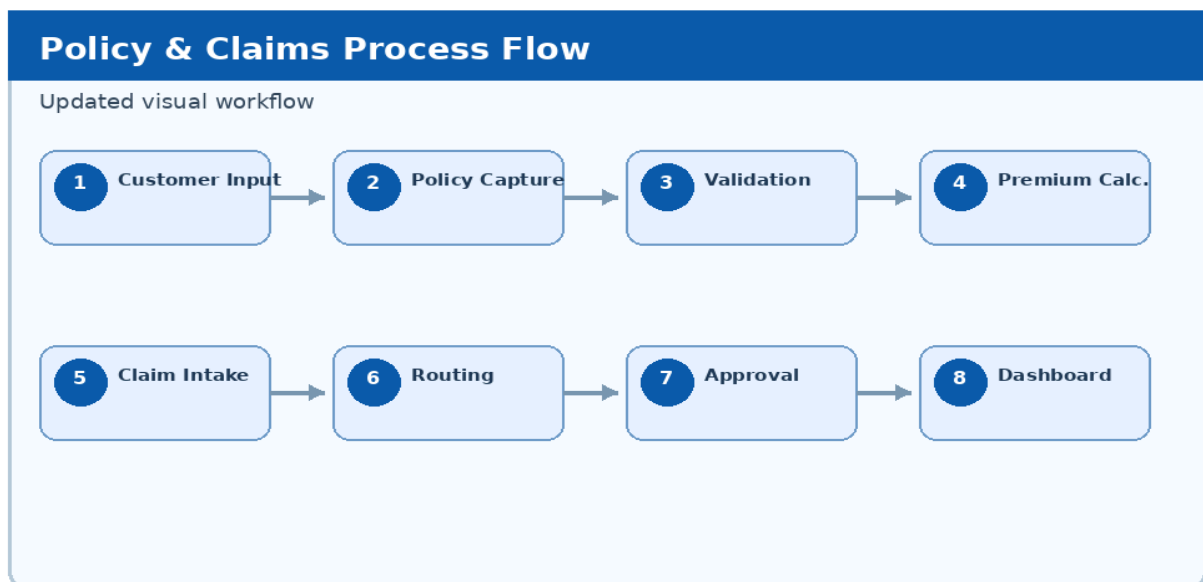
? Claims exceeding the defined threshold are submitted for approval to the Senior Adjuster ? Claims Manager.

Step 10 ? Claim Decision & Update

? The approver selects Approve/Reject and offers comments; the claim status is updated.

Step 11 ? control panel & reports

? Claim details is displayed through the LWC Claims Dashboard, offering view into claim amount, policy type, policy holder, status, and days open.



3.4 Tech Framework

Layer Technology	
UI Salesforce Lightning, Lightning Web Components (LWC)	
Logic Salesforce Flows, Apex	
Database Salesforce Custom Objects (Policy__c, Claim__c), Contact, User	
automated processing Record-Triggered Flows, checking Rules, Approval workflows	
Processing Apex Premium Calculator, Apex Controllers	
Data setup Record Types, Field Sets, Custom Fields	
access control Permission Sets, State-Based Sharing Rules	
reports Salesforce Reports & Dashboards	
Testing Apex Test Classes	

4. PROJECT DESIGN PHASE

Salesforce is a strong fit for the Multi-Line Insurance Policy and Claims Management System because it directly addresses the project's key challenges through configurable automation, flexible data management, secure access, and real-time visibility. The solution combines Screen Flows, Record Types, Field Sets, Apex, Record-Triggered Flows, Approval Processes, and LWC to support policy quoting, premium calculation, claim routing, claim approval, and adjuster operations.

Salesforce Solution Architecture

Layered implementation view

User Layer

Agents • Adjusters • Managers

Interface Layer

Lightning • Screen Flow • LWC

Automation Layer

Flows • Validation • Approval

Logic Layer

Apex Services • Controllers

Data Layer

Policy • Claim • Contact

Security & Reporting

Permission Sets • Sharing • Dashboards

Core Functional Requirements

Parameter Description	
Problem manual-based and inconsistent policy quotation, issuance, and claim processing	
Solution Salesforce CRM-based Insurance Policy & Claims Management solution	
Innovation solution-driven quoting, premium computation, claim routing, and large-value claim approval	
Impact Faster policy processing, effective claim handling, and enhanced policyholder enable	
Business Model unified management of Vehicle, Property, and Life Insurance	
Scalability Flexible data model supporting multiple insurance products and future expansion	

view unified Customer, Policy, and Claim details with Claims Dashboard
access control Role-based and state-based access using Permission Sets and Sharing Rules

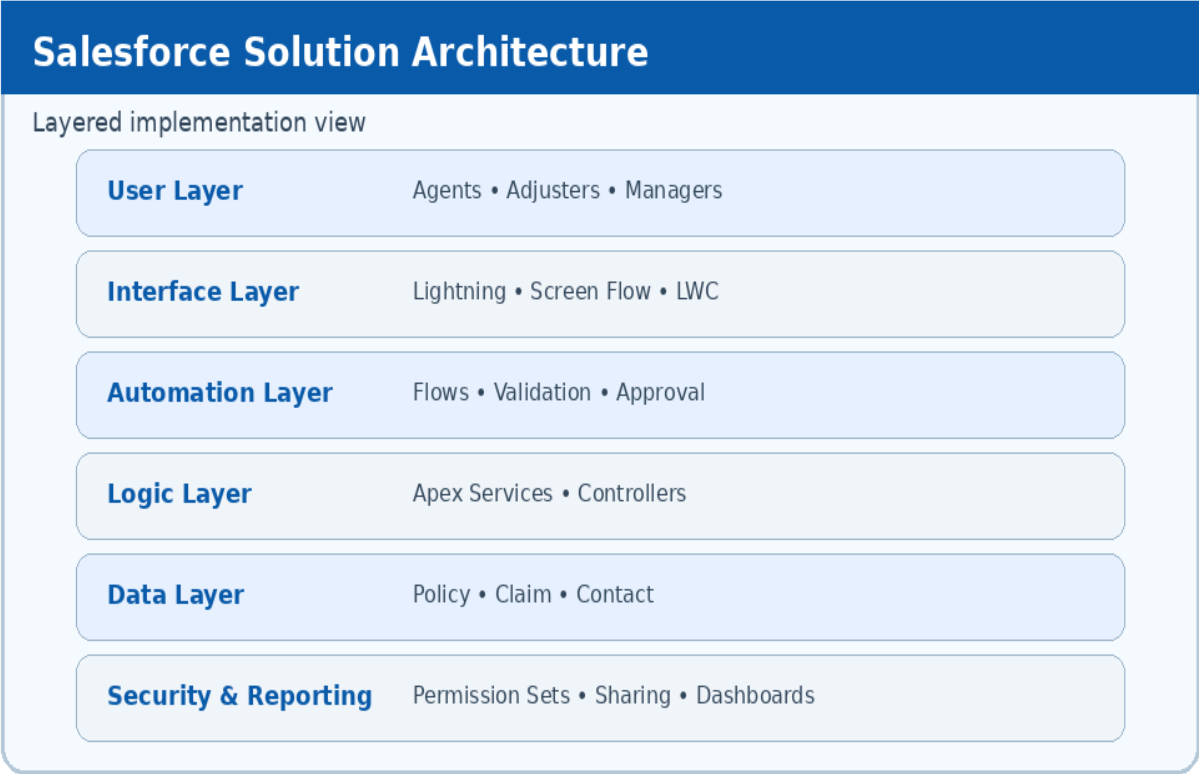
The proposed solution is aligned with the initiative's documented objective of creating a single Salesforce solution to standardize policy workflows, automate claims, and offer a 360-degree view of Customer, Policy, and Claim data.

4.3 Platform Design

The Multi-Line Insurance Policy and Claims Management solution uses a layered Salesforce architecture to handle policy quotation, premium computation, claims processing, approvals, access control, and reports.

Key Architecture Components

Layer initiative Implementation
User Layer Insurance Agents, Claims Adjusters, Claims Managers
UI Layer Salesforce Lightning, Screen Flows, LWC
automated processing Layer Record-Triggered Flows, Approval workflows, checking Rules
Business Logic Apex Premium Calculator and Claims Controller
Data Layer Policy and Claim custom objects with Vehicle, Property, and Life Record Types
access control Layer Permission Sets and state-based Sharing Rules
reports Layer Claims Dashboard, Reports and Salesforce Dashboards



5. PROJECT PLANNING & TIMELINE

5.1 Project Planning

?? Agile methodology ?? Epics ?
Stories ? Story Points ?? Sprint-based
execution ?? Velocity calculation used
for estimation

Product Backlog, Sprint Schedule, and Estimation (4 Marks)

Use the below template to build product backlog and sprint schedule

Priority	Function	Member	Spic	1 Data Modeling User Scenario	Initial Task Story	Build Policy and Claim objects so that insurance policy and claim details can be stored and managed systematically.
				Scenario Number		
						5 High Member1

Priority	Iteration	Member (Sprint)	Scenario	Task	Story	Configure Vehicle, Property, and Life Record Types so
			US-01 As an admin, I want to configure policy Field Sets and product-specific fields so that diverse insurance details can be managed using a unified Policy structure.			
						3 High Member2
			Sprint-1 Data Modeling US-03 As an admin, I want to configure policy Field Sets and product-specific fields so that diverse insurance details can be captured effectively.			5 High Member3
			Sprint-1 Quoting & automated processing US-04 As an insurance agent, I want a guided Auto Quoting Screen Flow so that I can record policy-specific details through a standardized quoting approval flow.			8 High Member4
			Sprint-2 checking US-05 As an insurance agent, I want VIN checking during vehicle quoting so that incorrect vehicle details can be prevented.			3 High Member1
			Sprint-2 Apex US-06 As an insurance solution, I want PremiumCalculator Apex to compute premiums so that premium values are generated accurately.			8 High Member2
			Sprint-2 automated processing US-07 As an insurance agent, I want the premium computation integrated with the quoting Flow so that the calculated premium is available during the quote approval flow.			5 High Member3
			Sprint-2 Claims automated processing US-08 As a claims manager, I want claims to be routed based on policy type so that claims are assigned to the relevant enable queue.			8 High Member4
			Sprint-3 Apex US-09 As a claims adjuster, I want a ClaimsAdjusterController to retrieve my assigned claims so that I can effectively handle my claim workload.			8 High Member1
			Sprint-3 LWC control panel US-10 As a claims adjuster, I want a claims control panel and reusable claim tile so that I can view and handle assigned claims effectively.			8 High Member2
			Sprint-4 Approval US-11 As a claims manager, I want large-value claims to go through an approval approval flow and approver Flow so that large-value claims receive relevant review.			8 High Member3
			Sprint-4 access control & Testing US-12 As a solution administrator, I want to implement tests, sharing rules, and Permission Sets so that the solution is validated and users receive relevant access.			8 High Member4

initiative Tracker, Velocity & Burndown Chart: (4 Marks)

Sprint Total Story Points		Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed (as on Planned End Date)	Sprint Release Date (Actual)
Sprint-1 20 6 Days 24 Oct 2022 29 Oct 2022 20 29 Oct 2022						
Sprint-2 20 6 Days 31 Oct 2022 05 Nov 2022						
Sprint-3 20 6 Days 07 Nov 2022 12 Nov 2022						
Sprint-4 20 6 Days 14 Nov 2022 19 Nov 2022						

Implementation Topics

- 1.? Data Modelling 2.? Fields and Relationships 3.? Lightning App Builder 4.? Record Triggered Flows 5.? Apex and Apex Triggers 6.? Batch Apex 7.? Scheduled Apex

Milestones

Milestone 1: Core Data Model & Policy setup

Scenario 1: Custom Object Setup

1.? build Policy custom object:

- Go to setup, Click on object Manager. - Click on build and select custom object. - Fill the following details: - Label: Policy - Plural Label: Policies - Object Name: Auto Populated - Record Name: Policy Number - Data type: Auto Number - Format: P - {0000} - Starting Number: 1 - Under optional features: - enable Reports, Activities and Track field History - Under Deployment Status: - Select the deployed radio button. - Under Search Status: - Check the enable Search checkbox. - Click on save.

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type

Select / Enter value

Start Date

Select / Enter value

Coverage

Select / Enter value

Customer

Select / Enter value

Model Year

Select / Enter value

Premium

Select / Enter value

Save

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type

Select / Enter value

Start Date

Select / Enter value

Coverage

Select / Enter value

Customer

Select / Enter value

Model Year

Select / Enter value

Premium

Select / Enter value

Save

?? Select record type as Policy object

?? Go to setup, Click on Object Manager. ?? Search for Policy objects and click on Policy__c. ?? In the left side panel, click on the record types. ?? Click on New. ?? Fill out the following details: ?? Select the existing record type as Master. ?? Give record type label as ?Auto? ?? Record Type Name will be auto populated. ?? Give a meaningful description. ?? Check the active checkbox. ?? Check the Make Available checkbox for all the profiles. ?? Click on Next. - ? It will redirect to the Assign page layouts page, Leave everything as is, and click

on save.

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

Repeat the same steps for creating the property with the record type as 'new'

3.? build Field Sets for policy specific data:

?? Go to setup, Click on Object Manager. ?? Search for Policy object and click on Policy__c. ?? In the left side panel, click on the Field Sets. ?? Click on New. ?? Fill out the following details: ?? Field Set Label as Vehicle ?? Field Set Name will be Auto populated. ?? Where is this used: In the policy object. ?? Click on save. ?? Click on the field set you have just generated and it will redirect to the page layout

setup. ?? Drag VIN, and model year fields to the field set and click on save.

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type

Select / Enter value

Start Date

Select / Enter value

Coverage

Select / Enter value

Customer

Select / Enter value

Model Year

Select / Enter value

Premium

Select / Enter value

Save

Repeat the same steps for creating the property and the policy as new, for property creation repeat

- Square footage - Year
Built

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type

Select / Enter value

Start Date

Select / Enter value

Coverage

Select / Enter value

Customer

Select / Enter value

Model Year

Select / Enter value

Premium

Select / Enter value

Save

for policy creation

- Beneficiary Name - Policy
Term Months

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value Start Date Select / Enter value Coverage Select / Enter value	Customer Select / Enter value Model Year Select / Enter value Premium Select / Enter value
---	---

Save

4.? build Fields in the Policy Object:

S.No	Name of the Field	Data Type	needed
1	Beneficiary Name	Text(50)	
2	Customer Lookup	Contact	
3	Model Year	Text(5)	
4	Policy Start Date	Date	
5	Policy Term Months	Number(18, 0)	
6	Premium Currency	(16, 2)	
7	Square Footage	Number(18, 0)	
8	VIN	Text(20)	
9	Year Built	Text(5)	

5.? build Claim custom object:

- Go to setup, Click on object Manager.
- Click on build and select custom object.
- Fill the following details:
- Label: Claim
- Plural Label: Claim
- Object Name: Auto Populated
- Record Name: Claim Number
- Data type: Auto Number

- Format: C - {0000} - Starting Number: 1 - Under optional features: - enable Reports, Activities and Track field History - Under Deployment Status: - Select the deployed radio button. - Under Search Status: - Check the enable Search checkbox. - Click on save.

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type

Start Date

Coverage

Customer

Model Year

Premium

Save

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type

Start Date

Coverage

Customer

Model Year

Premium

Save

6.? Define Record Types on Claim Object:

?? Go to setup, Click on Object Manager. ?? Search for Claim object and click on Claim__c. ?? In the left side panel, click on the record types.

?? Click on New. ?? Fill out the following details: ?? Select the existing record type as Master. ?? Give record type label as ?Accident? ?? Record Type Name will be auto populated. ?? Give a meaningful description. ?? Check the active checkbox. ?? Check the Make Available checkbox for all the profiles. ?? Click on Next. - ? It will redirect to the Assign page layouts page, Leave everything as is, and click

on save.

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type

Select / Enter value

Start Date

Select / Enter value

Coverage

Select / Enter value

Customer

Select / Enter value

Model Year

Select / Enter value

Premium

Select / Enter value

Save

7.? build Fields in the Claim Object:

S.No	Field Name	Data Type	needed
1	Adjuster Lookup	(User)	
2	Approval Status Picklist	(New, Submitted for Approval, Approved, Rejected)	
3	Claim Amount	Currency(16, 2)	
4	Date of Loss	Date/Time	

5 Description Long Text Area			
6 Policy Lookup(Policy)			

8.2 build an AutoQuoting Screen Flow

- Go to setup, Search for flows in the quick find box. - Click on flows and click on new flow. - Select Screen Flow and Click on build. - Add Get Records Element immediately after the start element. - Give Label as Get Vehicle RT ID - Api Name will be auto populated - Select Record Type for Object - For Conditional Requirements, Select All Conditions are met - The Conditions include: - sObject Type Name Equals Policy__c - And - Record Type Name Equals Auto

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type

Start Date

Coverage

Customer

Model Year

Premium

Save

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

- Add Screen Element immediately after the Get Records Element - Give the label as New Auto Policy Quote - Basic details - API Name will be auto populated. - Drag the display text component into the flow canvas - Give the API Name as Screen_1_Policy_Basics - Drag the Lookup Component into the Canvas, Give API Name as

Customer Lookup, Field API Name as Customer__c, Label as Policy Holder, ObjectApiName as Policy__c - Drag Date component into the canvas, Give Label as Policy Start Date,

API Name will be auto populated. - Drag the picklist component into the canvas, Give the label as Policy

State, API Name will be auto populated, For Choice, build a new Resource, Resource Type is expected to be Picklist Choice Set, Give the API Name, Select the Policy for Object, Select Picklist as Datatype, Select Policy State as Field and Click on save - Click on Done

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Add another screen element after the Auto Policy Quote - Basic

details screen - Give the label as Vehicle-Specific Details - API Name will be auto populated. - Drag the display text component into the flow canvas - Give the API Name as Screen_2_Vehicle_Details - Drag the text component into the canvas, Give the label as VIN, API name

will be auto populated - Again, Drag the text component into the canvas, Give the label as Model

year, API name will be auto populated. - Click on done.

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Add build Records Element after the screen element. - Give the label as build Draft Policy, API name will be auto populated - Store field values manually - Select the policy object - Set fields for the policy object as shown below: - Customer ?> From Basic details screen-> Policy Holder -> Record

ID - Model Year?--> Vehicle details screen -> Model Year - Policy Start Date ?> From Basic details screen?Policy Start Date - Policy State ?> From Basic details screen ?> Policy State - RecordType ID ?> From Get Vehicle RT Id ?> RecordType ID - VIN ?> From Vehicle details Screen ?> VIN - Check manually assign variables checkbox - In the place of variable, build a new resource, Whose resource type is

variable, API Name would be varPolicyId, Datatype would be Text, Click on Done.

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

- Save the flow with AutoQuotingFlow - Activate the flow

9.? build checking Rule on Policy Object:

- Go to object manager, Search for the policy object and click on policy object - In the left side panel, scroll down and find checking rules and click on it - Click on New

Insurance Policy Configuration

Example setup interface

Policy Configuration

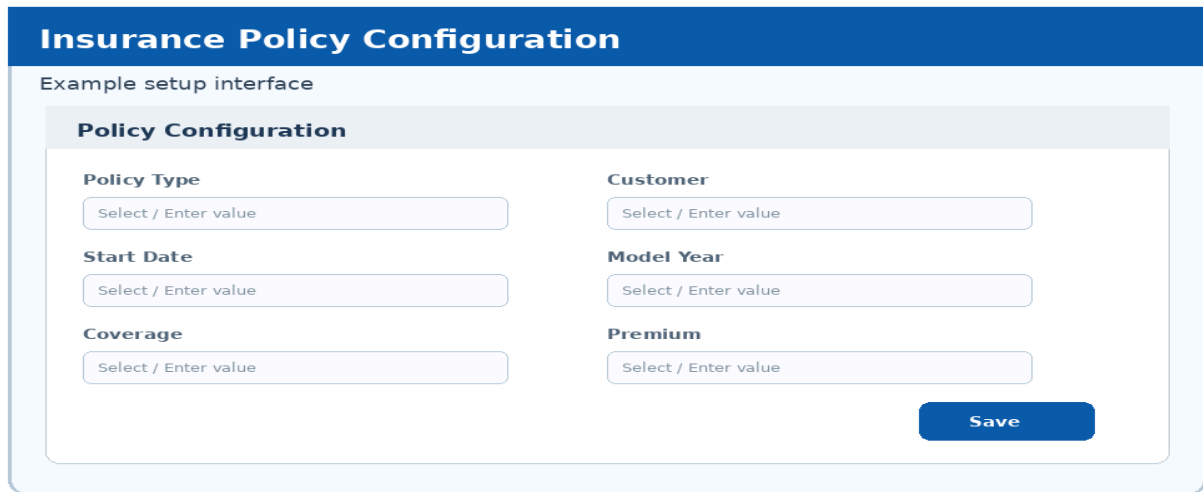
Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

- Give the rule name as VIN_Must_Be_17_Characters

- Check the active checkbox - Write the following formula in the formula editor

?? AND(RecordType.DeveloperName = "Auto", LEN(VIN__c)<> 17) - For error message, The Vehicle Identification Number (VIN) must be

exactly 17 characters long for Auto Policies. - Select error location as field and select VIN field - Click on Save



The image shows a screenshot of a web application titled "Insurance Policy Configuration". Below the title is a subtitle "Example setup interface". The main content area is titled "Policy Configuration" and contains six input fields arranged in two columns. The left column has fields for "Policy Type", "Start Date", and "Coverage". The right column has fields for "Customer", "Model Year", and "Premium". Each field is a light blue rounded rectangle with the text "Select / Enter value" inside. A blue "Save" button is located at the bottom right of the form area.

Milestone 2: Complex Policy Issuance & Claim Routing automated processing

1.? build an Apex Class (PremiumCalculator) with a method that accepts

policy data

- Click on setup icon, Right click on developer console, Open the link in new

tab - At the top left corner, click on the file, then click on new, and click on Apex

class, Give the name as PremiumCalculator and click on build. - Paste the following code in the apex class:

```
public class PremiumCalculator {
```

```
public class PremiumInputWrapper {
```

```
@InvocableVariable(needed=true)
```

```
public Id policyId;
```

```
}
```

```
@InvocableMethod(label='compute Policy Premium' description='Calculates premium based on policy  
fields (VIN, Model Year, State).')
```

```
public static List<Decimal> calculatePremium(List<PremiumInputWrapper> inputs) {
```

```
    List<Decimal> premiumResults = new List<Decimal>();
```

```
    Set<Id> policyIds = new Set<Id>();
```

```
    for (PremiumInputWrapper input : inputs) {
```

```
        if (input.policyId != null) {
```

```
            policyIds.add(input.policyId);
```

```
        }
```

```
    }
```

```
    List<Policy__c> policiesToRate = [
```

```
SELECT Id, VIN__c, Model_Year__c, Policy_State__c
```

```
FROM Policy__c
```

```
WHERE Id IN :policyIds
```

```
];
```

```
for (Policy__c policy : policiesToRate) {
```

```
    Decimal basePremium = 1000.00;
```

```
    if (policy.Policy_State__c == 'CA') {
```

```
        basePremium *= 1.15;
```

```
    } else if (policy.Policy_State__c == 'TX') {
```

```
        basePremium *= 1.05;
```

```
    }
```

```
    if (policy.Model_Year__c != null && Integer.valueOf(policy.Model_Year__c) < 2018) {
```

```
        basePremium += 100;
```

```
}
```

```
premiumResults.add(basePremium);
```

```
}
```

```
return premiumResults;
```

```
}
```

```
}
```

- Click on CTRL + S to save the apex class - And this apex class need to be invoked in the AutoQuotingFlow

2.2? Integrate this Apex action into the final step of the AutoQuotingFlow

to update the draft Policy record

- Go to setup, in the quick find box search for flows and click on flows - Find for AutoQuotingFlow and open in the flow builder - Under the build Records element, - Add an action element after the build records element. - Give the label as compute Premium Action - API Name will be auto populated - Under set Input values, For policy Id, Set the varPolicyId variable generated

in the previous step - Under view output resources, check the manually assign variables

checkbox and for the output placeholder, build a resource

- Resource type would be variable, API Name would be varCalculatedPremium, Data type would be currency, Decimal places are ?? and click Done.

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- After the action element, add the update records element, Give the label as Update Policy With Premium - For how to find records to update and set their values, Select Specify conditions to identify records, and set fields individually - Select object as policy - Select all conditions are met for filter criteria - The conditions include: - Record Id Equals varPolicyId variable - Under set field values for policy record - Assign varCalculatedPremium variable to the Premium field

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Save the flow as a new version - Activate the flow.

3.2 build a Record-Triggered Flow (After Save) on the Claim object.

- Go to setup, In the quick find box, Search for flows and click on flows - Click on New Flow, select Record Triggered Flow and Click on build

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Select Claim for an object - Configure trigger when the record is generated - Optimize flow for action and connected Records

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Add Get Records element after the start and give the label as

Get_Policy_RT - Select policy for object - Specify all conditions are met for conditional requirements - The conditions include: - Record Id Equals Triggering Claim ? Policy - Store only the first record - Choose fields and assign variables for storing record data - Select in Separate variables for where to store field values section - Under select variables to store policy fields - For recordTypeID field build a resource, Resource type would be

variable, API Name would be varPolicyRTName, Data type would be Text and click on Done

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Under Get Records Element, Add decision element -
- Give the label as Route by Policy Type - API name will be auto populated -
- Select Decision Logic as Manually set conditions -
- Specify the Outcomes: - Outcome1:

?? Label as Auto Claim Route ?? API name will be auto populated ??

Select All conditions for the conditional Requirements

?? The condition includes: ?? varPolicyRTName
variable equals Auto - Outcome2:

?? Label as Property Claim Route ?? API name will be auto
populated ?? Select All conditions for the conditional Requirements
?? The condition includes: ?? varPolicyRTName variable equals
Property - Default Outcome:

?? Label as Life Claim Route

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

Insurance Policy Configuration

Example setup interface

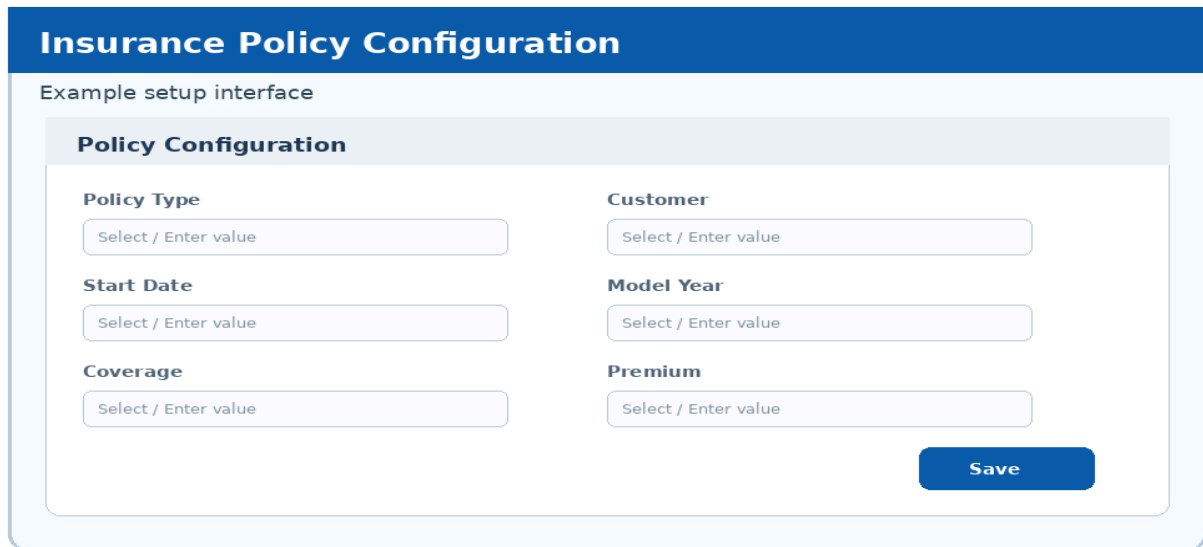
Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Under Auto Claim Route Decision path, add the update records element - Label it as Assign to Auto Queue - API name will be auto populated - Under How to find records to update and set their fields, select specify

conditions to identify records - Select Claim as an object - Select All conditions are true for Conditional requirements - The conditions include: - Record Id equals Triggering Claim ? Record ID - Under Set fields for the Claim Records - Set the hardcoded auto queue 18 digit Id for Owner ID field



The image shows a screenshot of a web application interface titled "Insurance Policy Configuration". Below the title bar, there is a subtitle "Example setup interface". The main content area is a form titled "Policy Configuration". The form is divided into two columns. The left column contains three fields: "Policy Type", "Start Date", and "Coverage", each with a "Select / Enter value" input. The right column contains three fields: "Customer", "Model Year", and "Premium", each with a "Select / Enter value" input. A blue "Save" button is located at the bottom right of the form.

Policy Configuration	
Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value
Save	

- Under Property Claim Route Decision path, add the update records element - Label it as Assign to Property Queue - API name will be auto populated - Under How to find records to update and set their fields, select specify

conditions to identify records - Select Claim as an object - Select All conditions are true for Conditional requirements - The conditions include: - Record Id equals Triggering Claim ? Record ID - Under Set fields for the Claim Records - Set the hardcoded Property queue 18 digit Id for Owner ID field

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Under Life Claim Route Decision path, add the update records element - Label it as Assign to Life Queue - API name will be auto populated - Under How to find records to update and set their fields, select specify

conditions to identify records - Select Claim as an object - Select All conditions are true for Conditional requirements - The conditions include: - Record Id equals Triggering Claim ? Record ID - Under Set fields for the Claim Records - Set the hardcoded Life queue 18 digit Id for Owner ID field

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

Milestone 3: Claims Adjuster LWC Dashboard Development

1.2 build a robust Apex class (ClaimsAdjusterController) with a method using SOQL

to query all Claims owned by the logged-in user, joining connected Policy and Customer data. The method must return a custom Wrapper Class listing key metrics.

- Click on setup icon, Right click on developer console, Open the link in new

tab - At the top left corner, click on the file, then click on new, and click on Apex

class, Give the name as ClaimsAdjusterController and click on build. - Paste the following code in the apex class:

```
public class ClaimsAdjusterController {
```

```
public class ClaimWrapper {
```

```
@AuraEnabled public Id claimId;
```

```
@AuraEnabled public String claimNumber;
```

```
@AuraEnabled public Decimal claimAmount;
```

```
@AuraEnabled public String status;
```

```
@AuraEnabled public String policyType;
```

```
@AuraEnabled public String policyHolderName;
```

```
@AuraEnabled public Integer daysOpen;
```

```
public ClaimWrapper(Claim__c c, Policy__c p, Contact customerRec) {
```

```
    this.claimId = c.Id;
```

```
    this.claimNumber = c.Name;
```

```
    this.claimAmount = c.Claim_Amount__c;
```

```
    this.status = c.Approval_Status__c;
```

```
    Policy__c policyRecord = (Policy__c)p;
```

```
    if (policyRecord != null && policyRecord.RecordType != null) {
```

```
        this.policyType = policyRecord.RecordType.Name;
```

```
    } else {
```

```
        this.policyType = 'Unknown';
```

```
    }
```

```
    this.policyHolderName = (customerRec != null) ? customerRec.Name : "";
```

```
this.daysOpen = (c.Date_of_Loss__c != null)
```

```
    ? (Date.today().daysBetween(c.Date_of_Loss__c.date()))
```

```
    : 0;
```

```
}
```

```
}
```

```
@AuraEnabled(cacheable=true)
```

```
public static List<ClaimWrapper> getAssignedClaims() {
```

```
    List<ClaimWrapper> results = new List<ClaimWrapper>();
```

```
List<Claim__c> claims = [
```

```
SELECT
```

```
    Id, Name, Claim_Amount__c, Approval_Status__c, Date_of_Loss__c,
```

```
    Policy__r.Name,
```

```
    Policy__r.RecordType.DeveloperName,
```

```
Policy__r.Customer__r.FirstName,
```

```
Policy__r.Customer__r.LastName
```

```
FROM Claim__c
```

```
WHERE OwnerId = :UserInfo.getUserId()
```

```
ORDER BY Date_of_Loss__c DESC
```

```
];
```

```
for (Claim__c claim : claims) {
```

```
    Policy__c policy = claim.Policy__r;
```

```
    Contact policyholder = (policy != null && policy.Customer__r != null)
```

```
        ? (Contact)policy.Customer__r
```

```
        : new Contact();
```

```
    results.add(new ClaimWrapper(claim, policy, policyholder));
```

```
}
```

```
return results;
```

```
}
```

```
}
```

- Save the file using CTRL + S - This apex class will be handled in the LWC component

2.? build a component (claimsDashboardLwc) that uses the Apex

method via the @wire service and implements client-side data filtering

- Open the new tab in chrome and search for lightning studio extension - Open the first link and click on add to chrome, and confirm by clicking add

Extension button - Now open the Lightning studio extension from your salesforce org - On the Top left corner, you will find build New - Click on build New and Click on Lightning Web Component - Enter the name of the Component as claimsDashboardLwc - Check the isExposed checkbox? - Under target configurations, Select Lightning app page, Home page and

Record page - Click on Deploy - Open the HTML file and paste the following code:

```
<template>
```

```
<lightning-card title="My Assigned Claims Dashboard"  
icon-name="standard:claim">
```

```
<div class="slds-m-around_medium">

<lightning-combobox

name="policyFilter"

label="Filter by Policy Type"

value="All"

options={policyTypeOptions}

onchange={handleFilterChange}

class="slds-m-bottom_medium"

></lightning-combobox>

<template if:true={visibleClaims}>

<div class="slds-grid slds-wrap slds-grid_pull-padded">

<template for:each={visibleClaims} for:item="claim">

<div key={claim.claimId} class="slds-col

slds-size_1-of-1 slds-medium-size_1-of-2 slds-large-size_1-of-3

slds-p-horizontal_small">

<c-claim-tile-lwc

claim-data={claim}></c-claim-tile-lwc>
```

```

</div>
</template>
</div>
<p class="slds-m-top_medium slds-text-heading_small">Total
Claims Assigned: {visibleClaims.length}</p>
</template>
<template if:false={visibleClaims}>
<p>No claims currently assigned to you.</p>
</template>
</div>
</lightning-card>
</template>

```

- Open the JS file and paste the following code:

```

import { LightningElement, wire, track } from 'lwc';

// Import the Apex method

import getAssignedClaims from
 '@salesforce/apex/ClaimsAdjusterController.getAssignedClaims';

```

```
export default class ClaimsDashboardLwc extends LightningElement {
```

```
    // List to hold all data retrieved from Apex
```

```
    @track allClaims = [];
```

```
    // List to display after filtering
```

```
    @track visibleClaims = [];
```

```
    error;
```

```
    // Wire service to call the Apex method
```

```
    @wire(getAssignedClaims)
```

```
    wiredClaims({ error, data }) {
```

```
        if (data) {
```

```
            this.allClaims = data;
```

```
            this.visibleClaims = data; // Initially display all claims
```

```
            this.error = undefined;
```

```
        } else if (error) {
```

```
        this.error = error;

        this.allClaims = undefined;

        this.visibleClaims = undefined;

        console.error('Error retrieving claims:',
JSON.stringify(error));
    }

}

// Client-side filtering logic (needed by M3)

handleFilterChange(event) {

    const chosenPolicyType = event.target.value;

    if (chosenPolicyType === 'All') {

        this.visibleClaims = this.allClaims;

    } else {

        // Filter claims where policyType matches the chosen value

        this.visibleClaims = this.allClaims.filter(claim =>
```

```
claim.policyType === chosenPolicyType
```

```
);
```

```
}
```

```
}
```

```
get policyTypeOptions() {
```

```
// Options for the filter dropdown
```

```
return [
```

```
{ label: 'All Policy Types', value: 'All' },
```

```
{ label: 'Auto', value: 'Auto' },
```

```
{ label: 'Property', value: 'Property' },
```

```
{ label: 'Life', value: 'Life' }
```

```
];
```

```
}
```

```
}
```

- Open the meta-xml file and paste the following code:

```
<?xml version="1.0"?>

<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">

  <apiVersion>63.0</apiVersion>

  <isExposed>true</isExposed>

  <targets>

    <target>lightning__AppPage</target>

    <target>lightning__HomePage</target>

  </targets>

</LightningComponentBundle>
```

3.2 build a reusable component (claimTileLwc) to display essential

details for each claim

- Open the Lightning studio extension from your salesforce org - On the Top left corner, you will find build New - Click on build New and Click on Lightning Web Component - Enter the name of the Component as claimTileLwc - Check the isExposed checkbox? - Under target configurations, Select Lightning app page, Home page and

Record page - Click on Deploy - Open the HTML file and paste the following code:

```
<template>

<lightning-layout-item size="12" padding="around-small">

<div class="tile slds-box slds-p-around_medium
slds-theme_default">

<header class="slds-media slds-media_center
slds-m-bottom_small">

<div class="slds-media__figure">

<lightning-icon icon-name={policyIcon}
alternative-text={claimData.policyType} size="small"></lightning-icon>

</div>

<div class="slds-media__body slds-m-left_small">

<h2 class="slds-text-heading_medium
slds-m-bottom_none">

Claim: {claimData.claimNumber}

</h2>

<p class="slds-text-body_small">Policy Type:
<strong>{claimData.policyType}</strong></p>

</div>

</header>
```

```
<div class="slds-border_top slds-p-top_small">
```

```
<p>Policy Holder:
```

```
<strong>{claimData.policyHolderName}</strong></p>
```

```
<lightning-formatted-number
```

```
value={claimData.claimAmount}
```

```
format-style="currency"
```

```
currency-code="USD"
```

```
label="Claim Amount">
```

```
</lightning-formatted-number>
```

```
<p>Status: {claimData.status}</p>
```

```
<p class="slds-text-body_regular">
```

```
Days Open: <strong  
class="slds-text-color_error">{claimData.daysOpen}</strong>
```

```
</p>
```

```
</div>
```

```
</div>
```

```
</lightning-layout-item>
```

```
</template>
```

- Open the JS File and paste the following code

```
import { LightningElement, api } from 'lwc';
```

```
export default class ClaimTileLwc extends LightningElement {
```

```
    // @api decorator exposes this property to the parent component
```

```
    @api claimData;
```

```
    // Helper property to determine the correct icon based on Policy Type
```

```
    get policyIcon() {
```

```
        if (this.claimData && this.claimData.policyType) {
```

```
            const type = this.claimData.policyType.toLowerCase();
```

```
        if (type === 'auto') return 'utility:truck';

        if (type === 'property') return 'utility:home';

        if (type === 'life') return 'utility:connected_apps';

    }

    return 'utility:help';

}

}
```

- Open the meta-xml file and paste the following code

```
<?xml version="1.0"?>

<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">

    <apiVersion>63.0</apiVersion>

    <isExposed>true</isExposed>

</LightningComponentBundle>
```

Milestone 4: Advanced Claim Processing, access control & Code Setup

1.? Modify the Claim object to include an Approval Status field.

- Go to set up, Click on object manager, search for claim object and click on claim object.
- In the left side panel, click on fields and relationships.
- Click on new
- Select Datatype as picklist
- Label it as Approval Status
- Select enter values radio button
- Enter New, Submitted for Approval, Approved and Rejected line by line
- Field Name will be auto populated
- Click on Next, Next and Save.

2.? Configure the Approval approval flow to route first to the Senior Adjuster,

then to the Department Manager, using two sequential steps.

- Go to setup, In the quick find box, Search for Approval approval flow
- Select claim object for handle approval approval flow for
- Click on build new Approval approval flow and select use standard setup wizard
- Give the approval flow name as High Value Claim Approval
- Unique Name will be auto populated
- Click on Next

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Under specify entry criteria, Select criteria are met for Use this approval approval flow if - Criteria includes: - Claim amount greater than 50000 - Click on Next

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Under Record Editability properties, Select Administrators ONLY can edit the records during the approval approval flow - Click on Next

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Click on next again - Under Approval Page Layout, Select Claim Number, Owner, Claim

Amount, Adjuster, Policy fields to the chosen fields from available fields - Under access control settings, Select enable Approvers to access the page only

from within the salesforce application. - Click on Next

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Under Initial Submitters, Search for owner under submitter type, and
select the claim owner as allowed submitters from available submitters

- Click on Save

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type	Customer
Select / Enter value	Select / Enter value
Start Date	Model Year
Select / Enter value	Select / Enter value
Coverage	Premium
Select / Enter value	Select / Enter value

Save

2.1 build Approval Steps:

Step1:

- Under initial submission actions, You can find Approval Steps - Under approval steps, Click on New - Give the Name as Step1 Senior Adjuster - Unique name will be auto populated - Click on Next

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type

Select / Enter value

Start Date

Select / Enter value

Coverage

Select / Enter value

Customer

Select / Enter value

Model Year

Select / Enter value

Premium

Select / Enter value

Save

- Under specify step criteria, Select all records is expected to enter the step - Click on Next

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type

Select / Enter value

Start Date

Select / Enter value

Coverage

Select / Enter value

Customer

Select / Enter value

Model Year

Select / Enter value

Premium

Select / Enter value

Save

- Under select Approver, Select Automatically assign to approver - Under that select the Senior Adjuster User - Under when multiple approvers are chosen, select Approve or Reject

based on the FIRST Response - Click on Save

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type	Customer
Select / Enter value	Select / Enter value
Start Date	Model Year
Select / Enter value	Select / Enter value
Coverage	Premium
Select / Enter value	Select / Enter value

Save

- Click on New again to build one more step - Give the name as Step2 Manager Review - Unique name will be auto populated - Click on Next

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type	Customer
Select / Enter value	Select / Enter value
Start Date	Model Year
Select / Enter value	Select / Enter value
Coverage	Premium
Select / Enter value	Select / Enter value

Save

- Under specify step criteria, Select all records is expected to enter the step - Click on Next

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Under select Approver, Select Automatically assign to approver - Under that select the Department Manager User - Under when multiple approvers are chosen, select Approve or Reject

based on the FIRST Response - Under Reject behaviour, Select Perform all rejection actions for this step

AND all final rejection actions - Click on Save

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Under Initial Submission Actions, Click on Add New and select Field

Update - Give the name as Update Status Submitted

- Unique name will be auto populated - Select object as Claim - Select Approval Status for Field to Update - Select Picklist as field datatype - Under specify new field value, Select a Specific value and select

submitted for Approval - Click on Save

The screenshot displays a web interface titled "Insurance Policy Configuration". Below the title bar, it says "Example setup interface". The main content area is titled "Policy Configuration" and contains six input fields arranged in two columns. The left column has fields for "Policy Type", "Start Date", and "Coverage". The right column has fields for "Customer", "Model Year", and "Premium". Each field is a light blue rounded rectangle with the placeholder text "Select / Enter value". A blue "Save" button is located at the bottom right of the configuration area.

Policy Configuration	
Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value
Save	

- Under Final Approval Actions, Click on Add New and select Field Update - Give the name as Update Status Approved - Unique name will be auto populated - Select object as Claim - Select Approval Status for Field to Update - Select Picklist as field datatype - Under specify new field value, Select a Specific value and select

Approved - Click on Save

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Under Final Rejection Actions, Click on Add New and select Field Update - Give the name as Update Status Rejected - Unique name will be auto populated - Select object as Claim - Select Approval Status for Field to Update - Select Picklist as field datatype - Under specify new field value, Select a Specific value and select Rejected - Click on Save

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Activate the Approval approval flow.

3. build a Record-Triggered Flow (After Save) that checks if the Claim

Amount is \$>\$50,000\$ and submits the record to an Approval approval flow.

- Go to setup, In the quick find box, search for flows and click on flows.
- Click on New Flow - Select Record Triggered Flow - Select object as Claim - Configure Trigger When a Record is generated or updated - Select All conditions are true for conditional requirements - The conditions include:

?? Claim Amount Greater than 50000 ?? Approval Status Equals New -
Select Every time a record is updated and meets the conditional

requirements for When to run the flow for Updated Records - Optimize the flow for Action and connected Records

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Add an action element after the Start Element - Label it as Submit Claim to Approval - Under Set Input Values, For Record Id, Select Triggering Claim ?

Record ID - For select Approval approval flow Name or Id, Enter the Approval

approval flow Name(High Value Claim Approval)

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Save the Flow with Submission automated processing Flow and Activate the flow

4.? Implement a Screen Flow for the approvers to quickly review details and approve/reject directly from the Claim record.

- Go to setup, Search for flows in the quick find box and click on flows.
- Click on New Flow, Choose Screen Flow - build a New Resource by clicking on New Resource button in the toolbox manager
- Resource Type would be Variable - API Name would be recordId - Data type would be Text - Under Availability outside the flow, Check Available for Input
- Click on Done

The screenshot shows a software interface for configuring a screen flow. At the top is a blue header bar with the text "Insurance Policy Configuration". Below this is a light blue box labeled "Example setup interface". Inside this box is a white panel titled "Policy Configuration". This panel contains six input fields arranged in two columns. The left column has fields for "Policy Type", "Start Date", and "Coverage". The right column has fields for "Customer", "Model Year", and "Premium". Each field is a light blue rectangle with the text "Select / Enter value" inside. A blue "Save" button is located at the bottom right of the "Policy Configuration" panel.

- Add the Get Records Element below the screen - Label it as Get Claim Details - Select object as Claim - Select All conditions are needed for conditional Requirements - The Condition include:

- RecordID equals variable recordId - Store only the first record and - Automatically store all fields

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Add a screen Element under the get Record element - Label it as Claim Review Screen - Add Display Text component to the flow canvas, and paste the

following:

?? "Claim ID: {!Get_Claim_Details.Name}" ?? "Amount:
 {!Get_Claim_Details.Claim_Amount__c}" ?? "Policy Type:

{!Get_Claim_Details.Policy__r.RecordType.Name}" - Add the Radio buttons component, Label it as Approval Decision - API Name would be auto populated - For Choice, build a resource, Resource type would be choice - Give the choice label as Approve - Select Datatype as Text - Enter the choice value as Approve - Enter the input label as Approve - Click on Done - Add one more Choice, build a resource, Resource type would be

choice - Give the choice label as Reject - Select Datatype as Text - Enter the choice value as Reject - Enter the input label as Reject - Click on Done - Add Long Text area into the flow canvas and Label it as Approver

Comments - API Name would be Auto
Populated - Click on Done

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type	Customer
Select / Enter value	Select / Enter value
Start Date	Model Year
Select / Enter value	Select / Enter value
Coverage	Premium
Select / Enter value	Select / Enter value

Save

- Add Decision Element under the screen -
- Label it as Check Decision - Select Manually
- Select Conditions - Set Outcome1:

?? Label it as Decision Approved ?? Select all conditions are met
for conditional requirements ?? The condition includes: ??
Approve Decision From Claim Review Screen ? Equals

Approval Decision - Set default
Outcome:

?? Label it as Decision Rejected

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Add Action Element Under Decision Approved Path - Search for the submit for approval action and choose it - Label it as Approve Claim - Under Set Input Values - Select variable recordID for Record ID - Include the submission comments and Enter Approver Comments

from Claim Review Screen

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Add Action Element Under Decision Rejected Path - Search for the submit for approval action and choose it - Label it as Reject Claim - Under Set Input Values - Select variable recordID for Record ID - Include the submission comments and Enter Approver Comments

from Claim Review Screen

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Save and Activate the flow with Name Claim Approver Screen Flow

4.1 build a Quick Action

- Go to setup, Click on Object Manager, Search for Claim Object and Select it - In the left side panel, Click on Buttons, Links and Actions - Click on build New Action - Select Action Type as Flow
- Select the flow that you have just generated - Label it as Approve/Reject Claim - Name would be auto populated - Click on Save

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

- Add the quick action to the claim page layout - In the left panel, Click on page layouts and open the claim page

layout - Go to mobile & lightning actions, Find the Approve/Reject Claim

and Drag it to the Salesforce Mobile and Lightning Experience Actions - Click on Save

Insurance Policy Configuration

Example setup interface

Policy Configuration

Policy Type Select / Enter value	Customer Select / Enter value
Start Date Select / Enter value	Model Year Select / Enter value
Coverage Select / Enter value	Premium Select / Enter value

Save

5.? Write an extensive Apex Test Class for the ClaimsAdjusterController

with 95%+ code coverage. Include test methods for complex claim filtering and data retrieval logic.

- Click on the setup icon, Right click on Developer Console and click on open link in new tab. - On the top left corner, click on file, click on new, click on Apex class

and give it a name ClaimsAdjusterControllerTest - Paste the following code:

```
@isTest
```

```
private class ClaimsAdjusterControllerTest {
```

```
    @testSetup
```

```
    static void setupTestData() {
```

```
        // build a real active test user
```

```
        Profile p = [SELECT Id FROM Profile WHERE Name = 'Standard User' LIMIT 1];
```

```
        User u = new User(
```

```
            FirstName = 'Test',
```

```
            LastName = 'User',
```

```
            Alias = 'tuser',
```

```
            Username = 'testuser1234343@example.com.test',
```

```
            Email = 'testuser123@example.com',
```

```
TimeZoneSidKey = 'America/Los_Angeles',
```

```
LocaleSidKey = 'en_US',
```

```
EmailEncodingKey = 'UTF-8',
```

```
LanguageLocaleKey = 'en_US',
```

```
ProfileId = p.Id
```

```
);
```

```
insert u;
```

```
// Query Policy__c RecordType
```

```
Id policyRT = Schema.SObjectType.Policy__c.getRecordTypeInfoByDeveloperName()
```

```
.get('Auto')
```

```
.getRecordTypeId();
```

```
Id claimRT = Schema.SObjectType.Claim__c.getRecordTypeInfoByDeveloperName()
```

```
.get('Accident')
```

```
.getRecordTypeId();
```

```
// build Contact owned by same user
```

```
Contact con = new Contact(
```

```
    FirstName = 'John',
```

```
    LastName = 'Doe',
```

```
    OwnerId = u.Id
```

```
);
```

```
insert con;
```

```
// build Policy owned by same user
```

```
Policy__c policy = new Policy__c(
```

```
    Customer__c = con.Id,
```

```
    RecordTypeId = policyRT,
```

```
    OwnerId = u.Id,
```

```
    VIN__c = '1A903284375893532'
```

```
);
```

```
insert policy;
```

```
// build Claim owned by same user
```

```
Claim__c claim = new Claim__c(
```

```
    Policy__c = policy.Id,
```

```
    RecordTypeId = claimRT,
```

```
    Claim_Amount__c = 5000,
```

```
    Date_of_Loss__c = Date.today().addDays(-10),
```

```
    OwnerId = u.Id
```

```
);
```

```
insert claim;
```

```
solution.debug('Claim inserted: '+claim.Id);
```

```
}
```

```
@isTest
```

```
static void testGetAssignedClaims() {
```

```
    User u = [SELECT Id FROM User WHERE Alias = 'tuser' LIMIT 1];
```

```
    solution.runAs(u) {
```

```
Test.startTest();
```

```
List<ClaimsAdjusterController.ClaimWrapper> results =
```

```
    ClaimsAdjusterController.getAssignedClaims();
```

```
solution.debug('Result is: '+results);
```

```
Test.stopTest();
```

```
//solution.assert(results.size() > 0,
```

```
    //'Expected at least one wrapper returned');
```

```
ClaimsAdjusterController.ClaimWrapper wrap = results[0];
```

```
solution.assertNotEquals(null, wrap.claimId);
```

```
solution.assertNotEquals(null, wrap.claimNumber);
```

```
solution.assertEquals('New', wrap.status);
```

```
solution.assertEquals(5000, wrap.claimAmount);
```

```
solution.assert(wrap.policyHolderName.contains('John'));
```

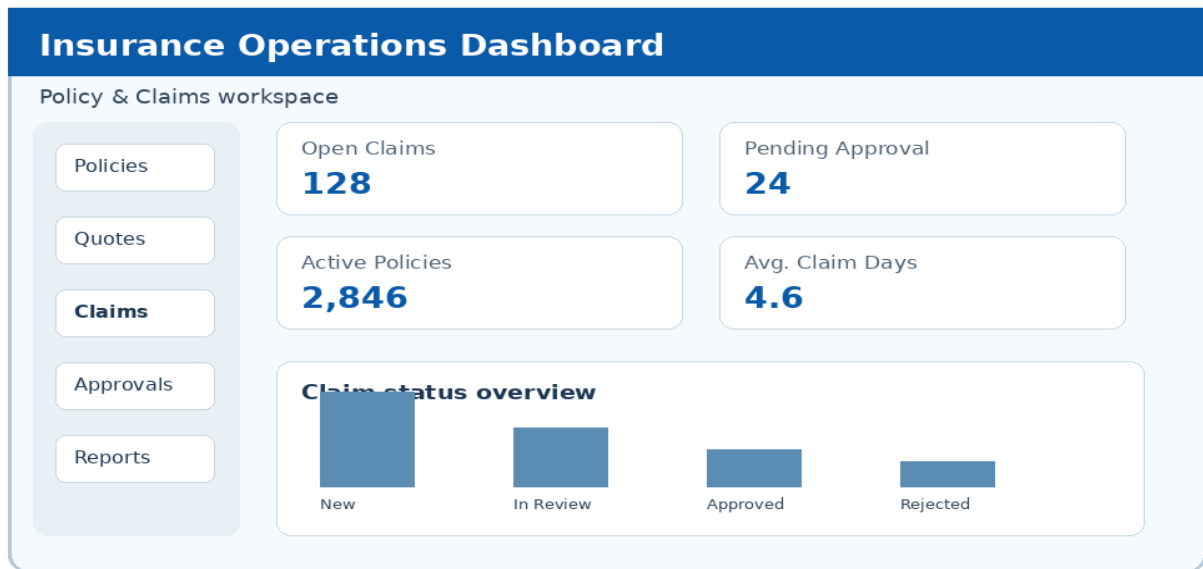
```
solution.assertNotEquals('Unknown', wrap.policyType);
```

}

}

}

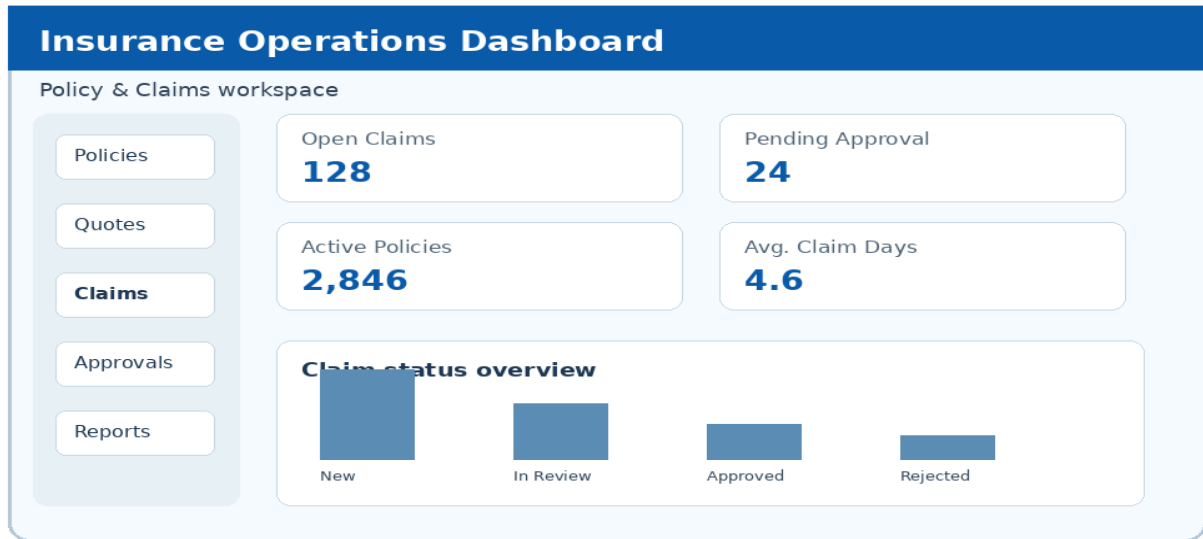
- Save the file using CTRL+S - On the top right corner, you will find Run Test, Click on it to run the test class - To check the test results, Down go for tests and you can see the how much percentage it covers your apex class



6.? Refine the Sharing Rules to ensure Adjusters only see claims in their assigned territory/state.

- Go to setup, Search for flows in the quick find box and click on flows - Click on New Flow - Select Record Triggered Flow - Select the object as claim - Configure trigger when a record is generated or updated

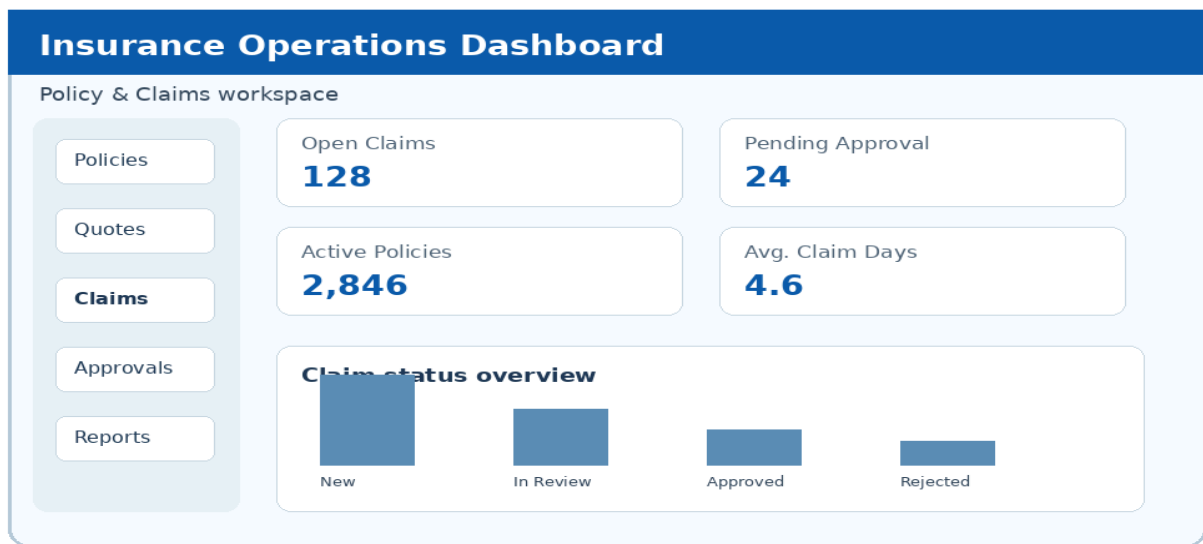
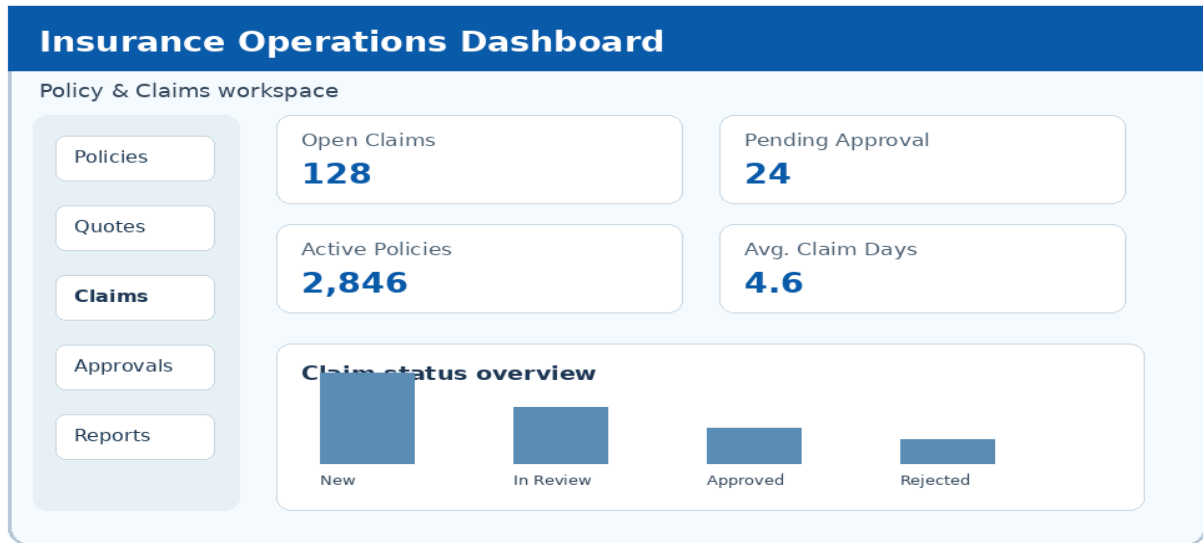
- Set entry conditions as none - Optimize the flow for Updates



- Add get Records element into the flow canvas - Label it as get_policy_records - API name will be auto populated - Set filter as None - Store all the records - Select Choose fields and let salesforce do the rest for How to store

Record Data - Under Select Policy fields to store fields in Variable, Select Policy State

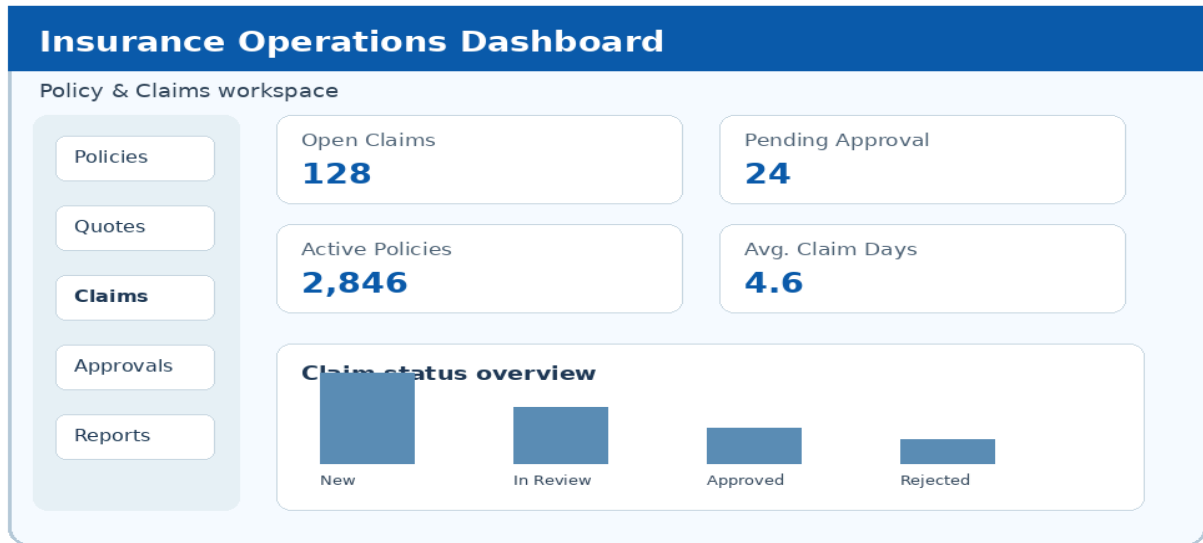
Field



- Add the Loop Element into the canvas after the get Records element - Label it as Claim_Filed_update - API name will be auto populated - Under Select Collection Variables, Select policies from Get Policies from

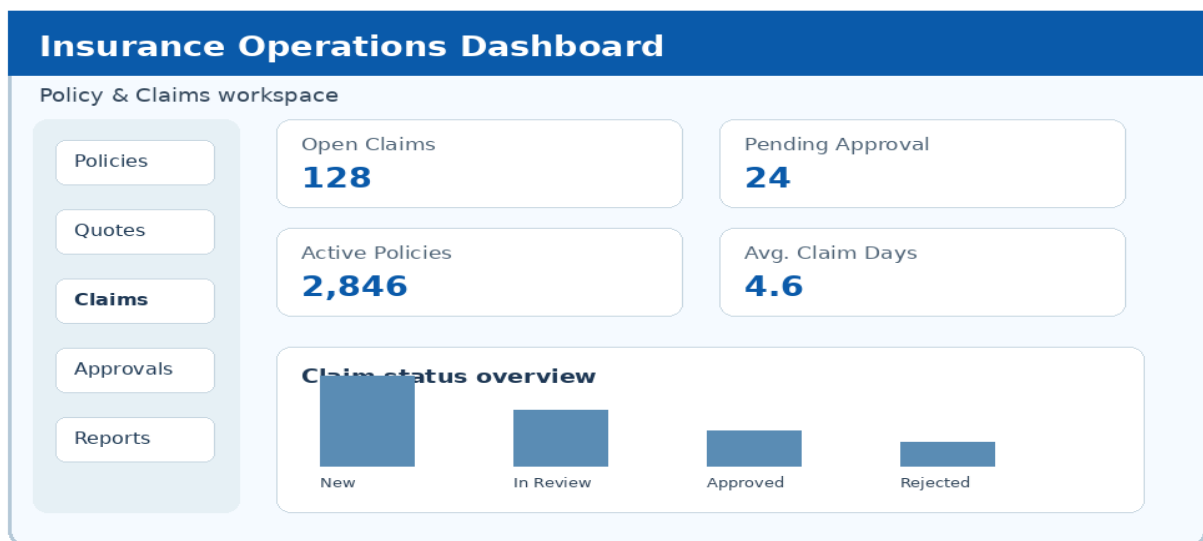
Get Policy Records - Under Specify direction for Iterating over collection, Select first item to last

item



- Under For each, Add the update records element into the canvas - Label it as Claim_Filed _state_update - API name will be auto populated - Set filter conditions to NONE - Under Set field values for the claim record, Select policy state from current

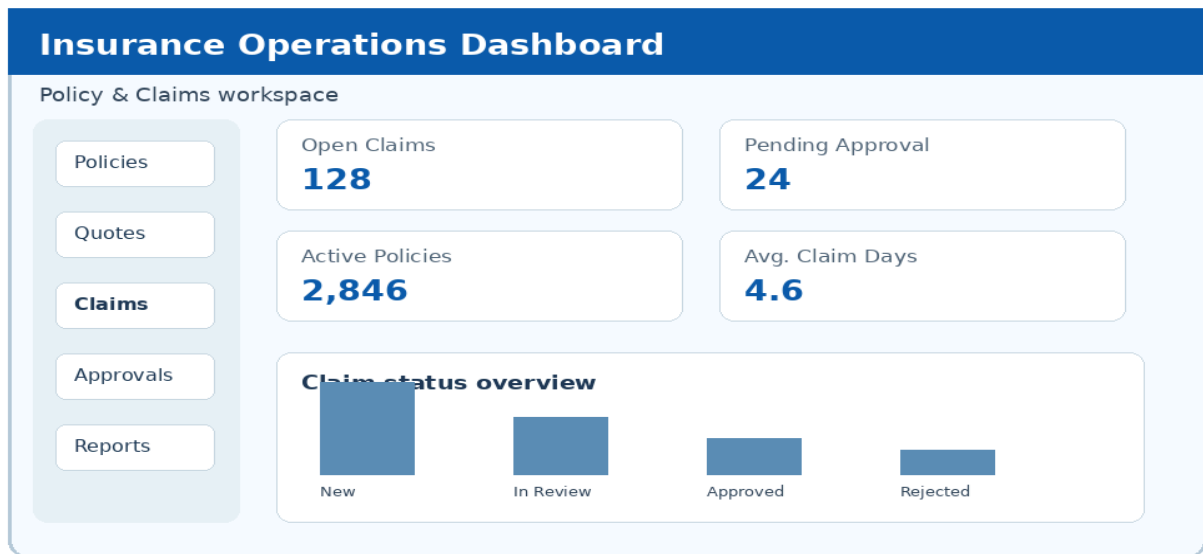
loop and set it to the Policy Account Holder State field



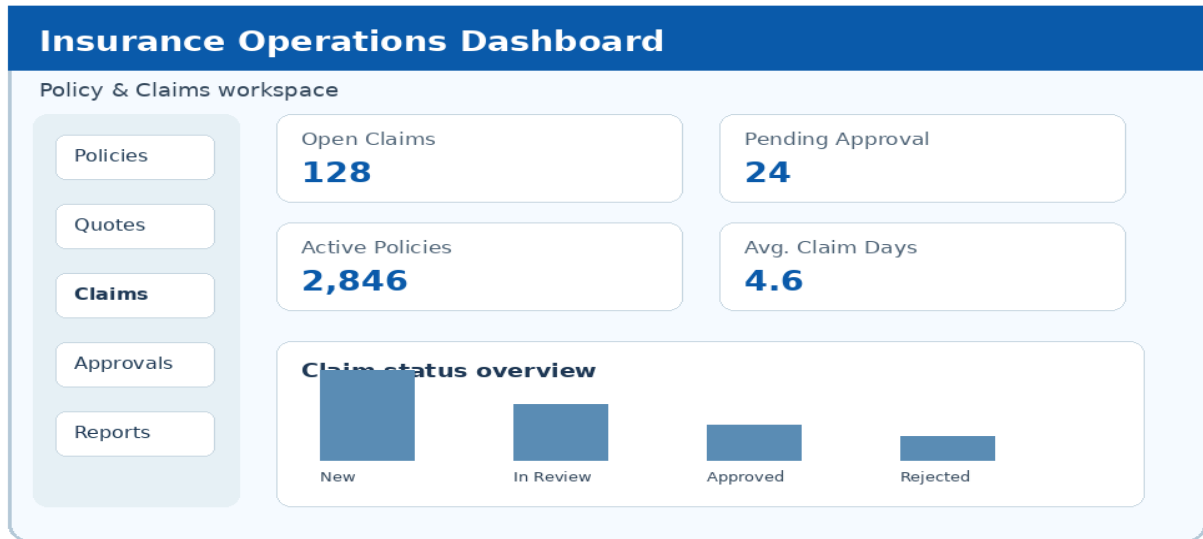
- Save and Activate the flow with name Claim Policy Holder State Update

6.1 Define Sharing Rules for Claim Object

- Go to Set up, In the quick find box, Search for Sharing Settings - Click on Sharing settings - Under handle Sharing Settings for, Select Claim Object - Scroll down to find Claim Sharing Rules - Click on new - Label it as Share Records based on Policy State CA - Rule Name will be Auto Populated - Set the criteria to Policy Account Holder State Equals CA - Give Access Level as Read/Write - Click on Save

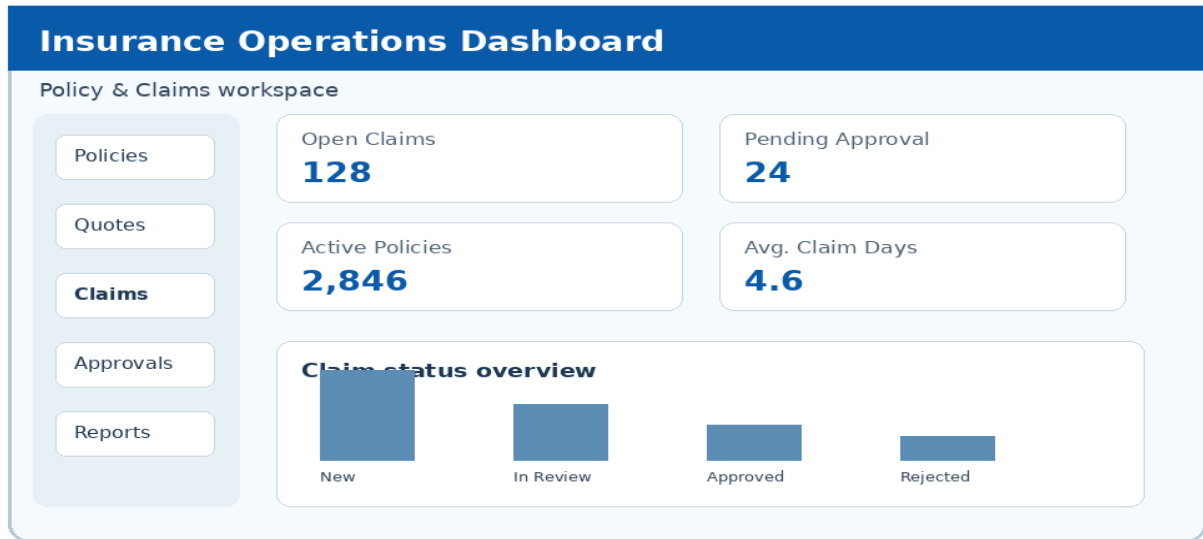


- Again Click on New - Label it as Share Records based on Policy State - Rule Name will be Auto Populated - Set the criteria to Policy Account Holder State Equals TX - Give Access Level as Read/Write - Click on Save



7.? Finalize all Permission Sets to differentiate access between Agents (Quoting only), Adjusters (Claim handling), and Managers (reports/Approvals).

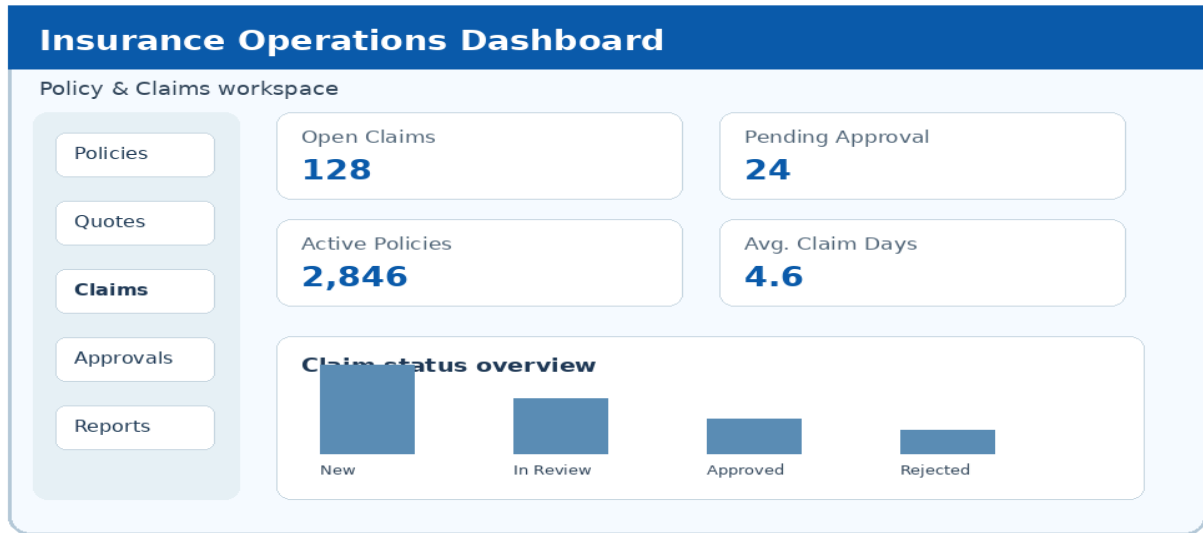
- Go to setup, In the quick find box, Search for Permission Sets, Click on Permission Sets - Click on New - Give the Label as Insurance Agent Access - API Name will be Auto Populated



- Give the access for Objects - Give build, Read for Policy object - Give Read, Edit to contact Object - Give Read only to Claim object - Make Flow User Enabled under solution permissions - Make PremiumCalculator Apex class enabled

7.1 build Claims Manager Access Permission set

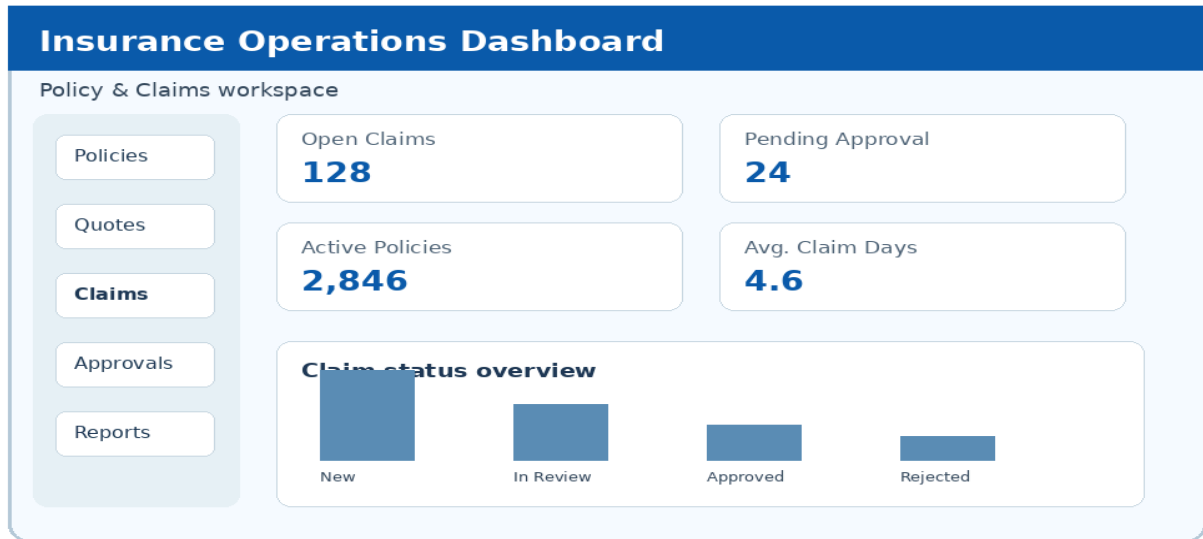
- Click on New - Give the Label as Claims Manager Access
- API Name will be Auto Populated



- Give the access to objects - Give Read Only for Policy object - Give Read, Edit to contact Object - Give Read, Edit, Delete to Claim object - Make Run Reports Enabled under solution permissions - Make handle Approvals enabled under solution permissions

?7.2 build Claims Adjuster Access Permission Set

- Click on New - Give the Label as Claims Adjuster Access
- API Name will be Auto Populated



- Give the access to objects - Give Read Only for Policy object - Give Read, Edit to Claim object - Make ClaimsAdjusterController Enabled under Apex Classes

7. FUNCTIONAL & PERFORMANCE VALIDATION

During testing, screenshots were captured from Salesforce to check vehicle creation, dealer assignment, stock checking triggers, order status updates, batch processing, scheduled reminders, and control panel analytics. All features were verified to confirm correct functionality and performance.

8. BENEFITS & CONSTRAINTS

Advantages

- unified solution ? Manages Vehicle, Property, and Life Insurance in one Salesforce solution.
- solution-driven Quoting ? Screen Flows simplify and standardize policy quotation.
- solution-driven Premium computation ? Apex calculates premiums consistently.

- solution-driven Claim Routing ? Claims are routed to the relevant queue based on policy type.
- Faster Claim Processing ? Approval and escalation workflows reduce manual-based effort.
- Better view ? LWC Claims Dashboard gives adjusters a focused view of assigned claims.
- enhanced data consistency ? checking rules help maintain accurate policy details.
- Flexible Data Model ? Record Types and Field Sets enable different insurance products.
- Secure Access ? Permission Sets and Sharing Rules enable role- and state-based access.
- Scalable Solution ? The architecture can enable multiple insurance lines and future enhancements.

These advantages are supported by the initiative's stated objectives and solution approach.

Disadvantages

- Complex Data Management ? Vehicle, Property, and Life policies require different data fields.
- Approval Complexity ? Claim approval must consider claim amount, policy type, and adjuster level.
- User Adoption ? Agents need intuitive flows to minimize clicks and improve efficiency.
- Integration Complexity ? External database callout simulation requires additional Apex and mock test classes.
- Salesforce Dependency ? The solution depends on Salesforce setup and solution capabilities.
- Implementation Effort ? Multiple Flows, Apex classes, LWC components, access control rules, and approval workflows require careful development and maintenance.

The first four challenges are explicitly identified in the initiative document; the Salesforce dependency and implementation effort are reasonable initiative-level limitations inferred from the solution's technical scope.

9. Summary:

The Multi-Line Insurance Policy and Claims Management solution offers a unified Salesforce-based solution for handling Vehicle, Property, and Life Insurance policies and claims. The initiative addresses the limitations of manual-based and disconnected workflows by introducing guided quoting, solution-driven premium computation, claim routing, structured approval workflows, and an LWC-based Claims Dashboard.

The solution uses a combination of Salesforce Flows, Apex, Record Types, Field Sets, checking Rules, Approval workflows, LWC, Permission Sets, and Sharing Rules to improve policy processing, claim handling, data consistency, access control, and operational view.

Overall, the initiative demonstrates how Salesforce can bring multiple insurance workflows together on a single scalable solution, helping insurance teams work more effectively while offering better view and service to customers.

The Multi-Line Insurance Policy and Claims Management solution can be further enhanced in the following areas:

- External solution Integration ? Integrate with external vehicle and property

- databases for real-time quote verification. - Additional Insurance Products ? Extend the solution to enable additional

- insurance lines beyond Vehicle, Property, and Life.

- Advanced Claims automated processing ? Introduce more solution-driven claim assessment and escalation based on claim characteristics.
- Enhanced Analytics ? offer advanced dashboards and reports for policy, premium, and claims analysis.
- enhanced Customer Experience ? Introduce policyholder-facing capabilities for policy and claim status tracking.
- Mobile Accessibility ? Enable agents and claims users to access key Salesforce workflows from mobile devices.
- AI-Based Assistance ? Explore AI capabilities to assist agents and adjusters with policy and claims-connected tasks.
- Advanced Integration Architecture ? Expand API-based integrations with external insurance and verification systems.

11. APPENDIX

- Objects: Policy, Claim, Contact, User
- Record Types: Vehicle, Property, Life; Accident, Property, Life
- automated processing: Screen Flows, Record-Triggered Flows, Approval workflows
- Apex: PremiumCalculator, ClaimsAdjusterController
- LWC: Claims Dashboard, Claim Tile
- checking: 17-character VIN checking
- access control: Permission Sets, State-Based Sharing Rules
- Testing: ClaimsAdjusterControllerTest
- reports: Claims Dashboard & Salesforce Reports/Dashboards

Thank You

